CASE STUDY DOCUMENT

Real-Time Chat Application

Socket.io Alternative in Java

(Spring WebSocket + STOMP + SockJS Integration)

ConnectHub

Connect Instantly. Chat Securely. Collaborate in Real Time.

Version 1.0 | 2026

1. Synopsis

ConnectHub is a full-stack Real-Time Chat Application built using Spring WebSocket with STOMP protocol. The system enables users to communicate in real time through individual direct messages and group channels, sharing text messages, images, files, and emoji reactions — all delivered with sub-100ms latency over persistent WebSocket connections managed by Spring's built-in STOMP message broker.

The platform supports persistent bi-directional communication through Spring's WebSocketMessageBrokerConfigurer, which registers STOMP message broker endpoints and maps WebSocket destinations to handlers. Users subscribe to room-level topics (/topic/room/{roomId}) and personal queues (/topic/user/{userId}) to receive real-time events: new messages, typing indicators, read receipts, emoji reactions, presence updates, and delivery status changes — all without polling.

ConnectHub is architectured as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, room/channel management, message lifecycle management, media and file management, user presence and status tracking, and notifications — with a dedicated WebSocket Handler layer (ChatWebSocketHandler + WebSocketConfig) implementing the real-time messaging core — all integrated through a Spring MVC website controller layer.

2. Detailed Requirements

2.1 General Requirements

- Users must register or log in (email/password or Google/GitHub OAuth) to access the chat platform.
- All WebSocket connections are authenticated via a JWT token passed as a STOMP connect header.
- Role-based access: User (standard chat participant), Room Admin (manages a specific room), and Platform Admin.
- All users can update their profile, avatar, and online status message.
- In-app push and email notifications are dispatched for mentions, missed messages, and system events.
- Platform Admin panel provides full user, room, and connection management with real-time analytics.

2.2 Authentication & Profile Requirements

- Register and log in via email/password or Google/GitHub OAuth.
- JWT token is issued on login and passed as a STOMP connect Authorization header to authenticate WebSocket connections.
- Update profile: full name, username, avatar image, and bio.
- Change password with current password verification.
- Search other users by username for initiating direct messages or adding to groups.

- Update online status: Online, Away, Do Not Disturb, or Invisible (appears offline).
- Last-seen timestamp is recorded on every WebSocket disconnect.

2.3 Messaging Requirements

- Send text messages in real time via WebSocket STOMP /app/chat.send endpoint; message is persisted and broadcast to /topic/room/{roomId}.
- Send image and file attachments: uploaded to S3 first via REST API, then the media URL is sent as a MESSAGE payload with type IMAGE or FILE.
- Reply to any message by setting the replyToMessageId field; the UI renders the original message as a quoted thread preview.
- Edit a sent message: updated content is broadcast to room subscribers via a MESSAGE_EDIT STOMP event.
- Delete a sent message: soft-deleted (isDeleted=true); a MESSAGE_DELETE event is broadcast so all recipients' UIs remove it.
- React to a message with an emoji: REACTION event is broadcast with senderId, messageId, and emoji.
- Scroll-based message pagination: REST API returns paginated message history (newest first); WebSocket delivers new messages in real time.
- Search messages within a room by keyword via REST API.

2.4 Room / Channel Requirements

- Create a Group Room (multi-user) or Direct Message (DM, always private, exactly 2 members).
- Join a group room via an invite link or by being added by a room admin.
- Leave a room at any time; last message and unread count are retained for history.
- View all rooms the user belongs to, sorted by lastMessageAt descending.
- View room members list with their roles and online status.
- View full message history for any room with infinite scroll (paginated REST).
- Room settings: name, description, avatar image, and max member limit.
- Unread message count is tracked per user per room using lastReadAt timestamp.

2.5 Room Admin Requirements

- Add members to a group room; assign roles (Admin or Member).
- Remove members from a group room.
- Change a member's role between Admin and Member.
- Mute a member (they can still read but cannot send messages).
- Delete any message in their rooms (moderation).
- Clear entire room message history.
- Pin a message to the top of the room for easy reference; unpin to remove.

2.6 Real-Time Signal Requirements

- Typing Indicator: when a user types, a TYPING_INDICATOR STOMP event (senderId, roomId, isTyping=true) is broadcast; cleared after 3 seconds of no keystroke.
- Read Receipts: when a user reads up to a message, a READ_RECEIPT STOMP event (readerId, roomId, upToMessageId) is broadcast to all room members.
- Delivery Status: messages transition through SENT (stored) → DELIVERED (recipient's WebSocket is active) → READ (read receipt received).
- Presence Updates: when a user connects, disconnects, or changes status, a PRESENCE_UPDATE STOMP event is broadcast to all rooms the user belongs to.
- Online indicator and last-seen timestamp are displayed for all users in a room.

2.7 Media & File Requirements

- Upload images (JPEG, PNG, GIF, WebP) and documents (PDF, DOCX, ZIP) up to 25MB per file.
- Image thumbnails are auto-generated server-side for preview in the chat window.
- Uploaded files are stored on AWS S3; download links are pre-signed S3 URLs with 24-hour expiry.
- View a room's shared media gallery showing all images and files exchanged in the room.
- Download any file attachment by clicking the download button in the chat.

2.8 Notification Requirements

- In-app notifications for: new message in a room where the user is offline, @mention in a message, room invite.
- Push notifications (Firebase FCM) for mobile users who are offline.
- Email notification for missed direct messages when user has been offline for more than 30 minutes.
- Notification badge count displayed in real time on the room list and navigation bar.
- Mark individual or all notifications as read from the notification centre.

2.9 Platform Admin Requirements

- View and manage all user accounts: suspend, reactivate, or permanently delete.
- View and manage all rooms: delete any room or message for policy violations.
- View active WebSocket connection count in real time on the admin dashboard.
- Access platform analytics: total users, messages per day, active rooms, file storage used.
- Send platform-wide broadcast messages to all connected users via STOMP.
- View audit logs of all admin actions.

3. Use Case Diagram

The diagram below captures all actors and use cases within the ConnectHub platform — spanning authentication, real-time messaging over WebSocket/STOMP, room management, typing indicators and read receipts, media uploads, presence tracking, notifications, room administration, and platform-wide oversight.

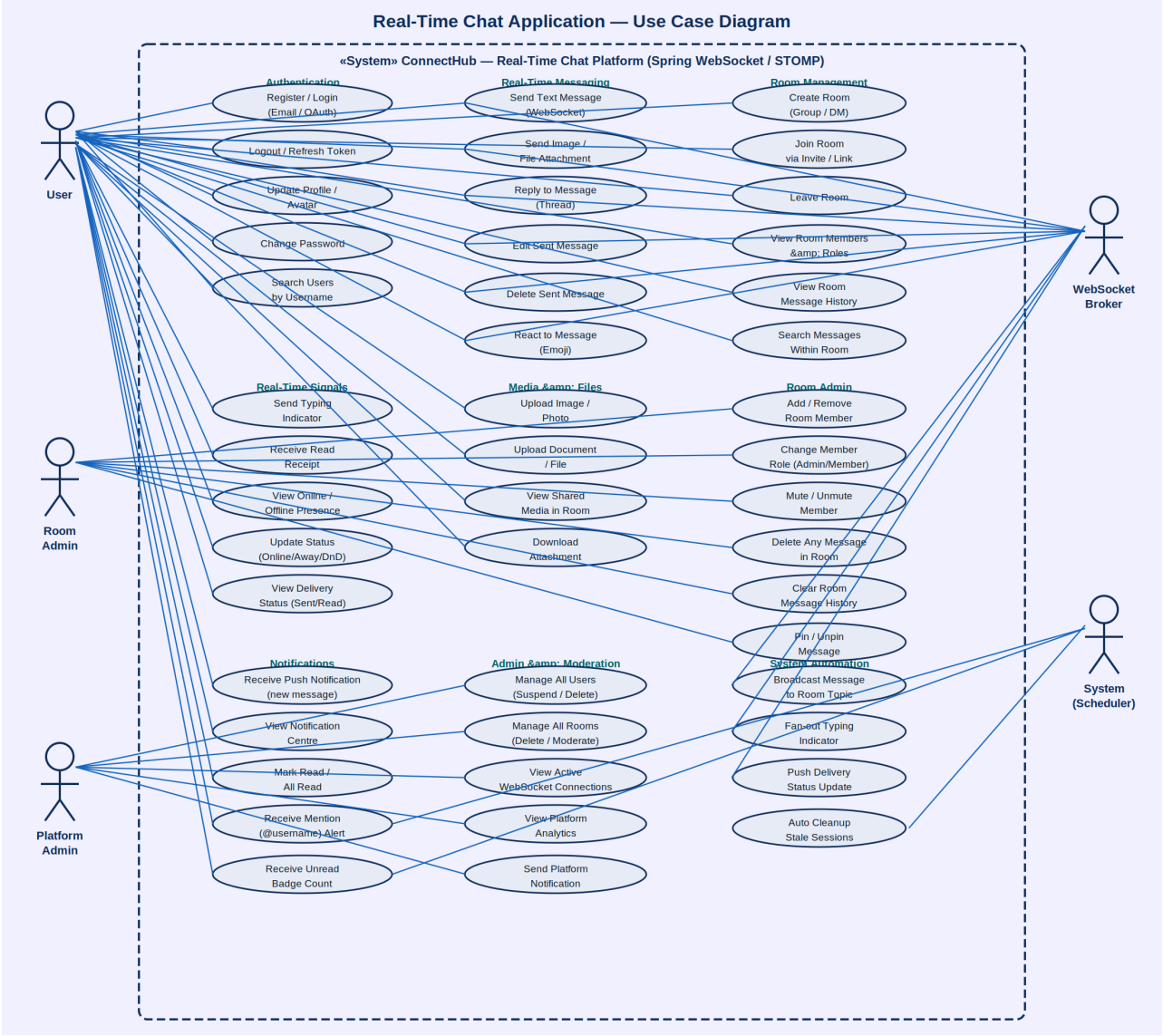

Figure 1: ConnectHub Real-Time Chat Application — Complete Use Case Diagram

3.1 Actors

Actor	Description
User	Registered chat participant who sends messages, joins rooms, and manages their profile and status
Room Admin	Group room creator or elevated member who manages room settings, members, and message moderation
Platform Admin	System administrator who manages users, rooms, connections, and platform analytics
WebSocket Broker	Spring STOMP message broker that routes frames between clients via topic subscriptions
System (Scheduler)	Automated component handling stale session cleanup, offline notification dispatch, and presence timeouts

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	User	Create account or authenticate via email or Google/GitHub OAuth
Update Profile / Avatar	User	Change username, full name, avatar image, or bio
Change Password	User	Update account password with current password verification
Search Users	User	Find other users by username to initiate a DM or add to a group
Update Online Status	User	Set status to Online, Away, Do Not Disturb, or Invisible
Send Text Message	User, WS Broker	Submit a text message via STOMP /app/chat.send; broker broadcasts to /topic/room/{id}
Send Image / File	User	Upload media via REST to S3; send URL as IMAGE/FILE type STOMP message
Reply to Message	User	Set replyToMessageId to thread a reply under any existing message
Edit Sent Message	User	Update own message content; MESSAGE_EDIT event broadcast to room subscribers
Delete Sent Message	User	Soft-delete own message; MESSAGE_DELETE event broadcast to room subscribers
React to Message	User	Send an emoji REACTION event for any message in the room

Use Case	Actor(s)	Description
Create Room (Group / DM)	User	Create a named group room or a private 2-person direct message channel
Join Room	User	Accept an invite link or be added by a room admin to a group room
Leave Room	User	Exit a room; unread count is preserved for history
View Room Members	User	See all members with their roles and real-time online status indicators
View Message History	User	Load paginated past messages via REST API on room open
Search Messages in Room	User	Full-text keyword search across a room's persisted message history
Send Typing Indicator	User, WS Broker	Broadcast TYPING_INDICATOR frame on keystroke; cleared after 3s inactivity
Receive Read Receipt	User, WS Broker	Broadcast READ_RECEIPT frame when user reads up to a specific message
View Presence Status	User	See live online/offline indicators and last-seen time for room members
View Delivery Status	User	See SENT / DELIVERED / READ status icons on own messages
Upload Image / Photo	User	Upload an image to S3; auto-thumbnail generated; URL sent as chat attachment
Upload Document / File	User	Upload a document (PDF, DOCX, ZIP) to S3; URL sent as FILE attachment
View Shared Media Gallery	User	Browse all images and files exchanged in a specific room
Download Attachment	User	Access a pre-signed S3 URL to download any file from the chat
Add / Remove Room Member	Room Admin	Invite or eject a user from a group room
Change Member Role	Room Admin	Promote or demote a member between Admin and Member roles
Mute / Unmute Member	Room Admin	Restrict a member from sending messages without removing them
Delete Any Message in Room	Room Admin	Moderate content by removing any message regardless of sender
Clear Room History	Room Admin	Permanently delete all messages in a room
Pin / Unpin Message	Room Admin	Pin an important message to the top of the room view

Use Case	Actor(s)	Description
Receive Push Notification	User, System	FCM push alert for new messages when user is offline on mobile
View Notification Centre	User	Browse all in-app alerts for mentions, room invites, and missed messages
Mark Notification Read	User	Dismiss individual or all unread notifications
Receive Mention Alert	User, WS Broker	Real-time alert when another user @mentions the user in a message
Manage All Users	Platform Admin	View, suspend, reactivate, or permanently delete any user account
Manage All Rooms	Platform Admin	Delete any room or individual message for policy violation
View Active Connections	Platform Admin	Monitor live WebSocket connection count and active session list
View Platform Analytics	Platform Admin	Access total users, messages per day, peak hour, and storage metrics
Send Platform Notification	Platform Admin	Broadcast a system-wide message to all connected users via STOMP
View Audit Logs	Platform Admin	Inspect timestamped record of all admin actions on the platform
Broadcast Message to Room	WS Broker	Route STOMP frame from /app/chat.send to /topic/room/{id} subscribers
Fan-out Typing Indicator	WS Broker	Broadcast TYPING_INDICATOR frames to all room subscribers
Push Delivery Status	WS Broker	Update DELIVERED status when recipient WebSocket session is confirmed active
Auto Cleanup Stale Sessions	System	Scheduler removes presence records for sessions that have not pinged in 60s
Receive Unread Badge Count	User	Real-time unread message counter updated via personal queue subscription

4. Class Diagrams — All Services

ConnectHub follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). A dedicated WebSocket Handler layer (ChatWebSocketHandler + WebSocketConfig) implements the STOMP real-time messaging core. The nine class diagrams below detail every service and handler in the platform.

4.1 Auth / User-Service

The Auth/User-Service manages all user identity operations. On successful login, a JWT token is issued and the client passes it as the STOMP connect Authorization header so the WebSocket handshake can verify the identity. The status field (ONLINE/AWAY/DND/INVISIBLE) is broadcast as a PRESENCE_UPDATE STOMP event whenever it changes. The lastSeenAt timestamp is written on every WebSocket disconnect event received by the ChatWebSocketHandler.

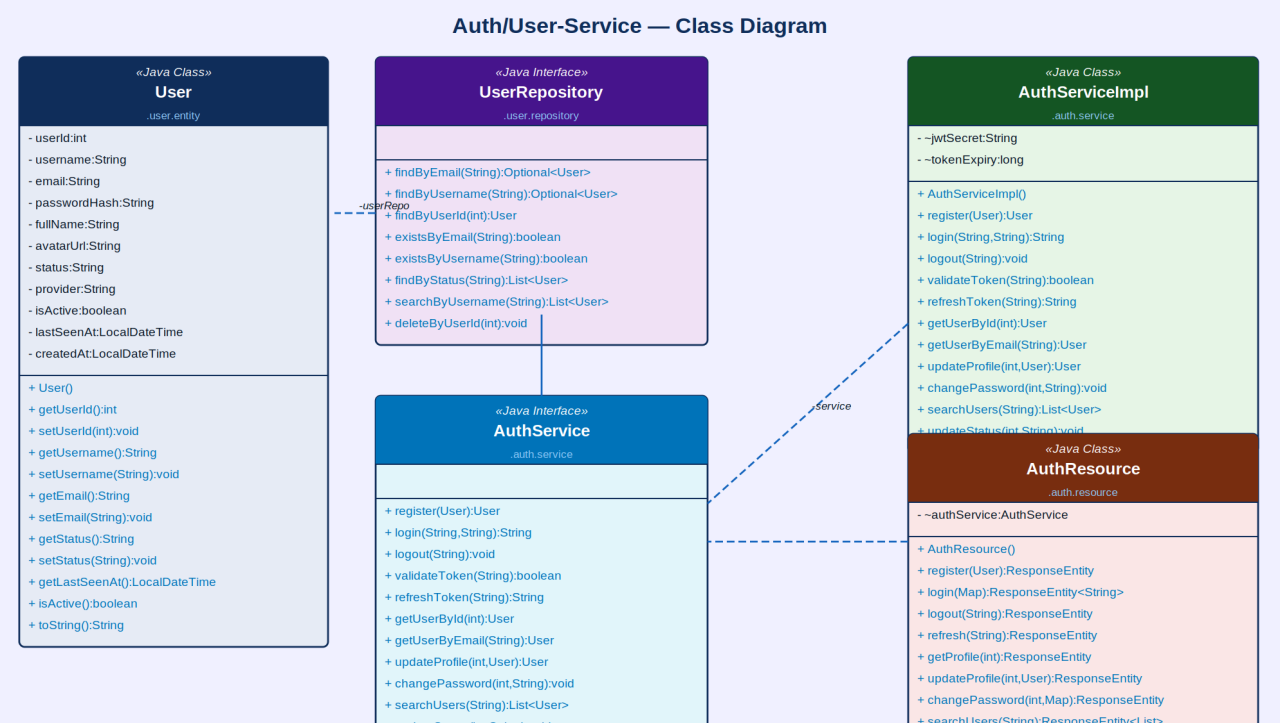

Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, username, email, passwordHash, fullName, avatarUrl, status (ONLINE/AWAY/DND/INVISIBLE), provider, isActive, lastSeenAt, createdAt
UserRepository	Java Interface	findByEmail(), findByUsername(), findByUserId(), existsByEmail(), existsByUsername(), findByStatus(), searchByUsername(), deleteByUserId()
AuthServiceImpl	Java Class	Implements register, login, JWT generation/validation, OAuth2 handling, profile update, password change, user search, status update, last-seen recording
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), getUserById(),

Component	Type	Responsibility
		updateProfile(), changePassword(), searchUsers(), updateStatus()
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/search, /auth/status

4.2 Room / Channel-Service

The Room/Channel-Service manages chat rooms and membership. Room types are GROUP (multi-user) and DM (exactly 2 members, always private). RoomMember records carry a role (ADMIN/MEMBER), a lastReadAt timestamp for unread count computation, and a isMuted flag. The updateLastRead operation is called whenever a READ_RECEIPT STOMP event is received, and the unread count is recomputed as the number of messages sent after lastReadAt.

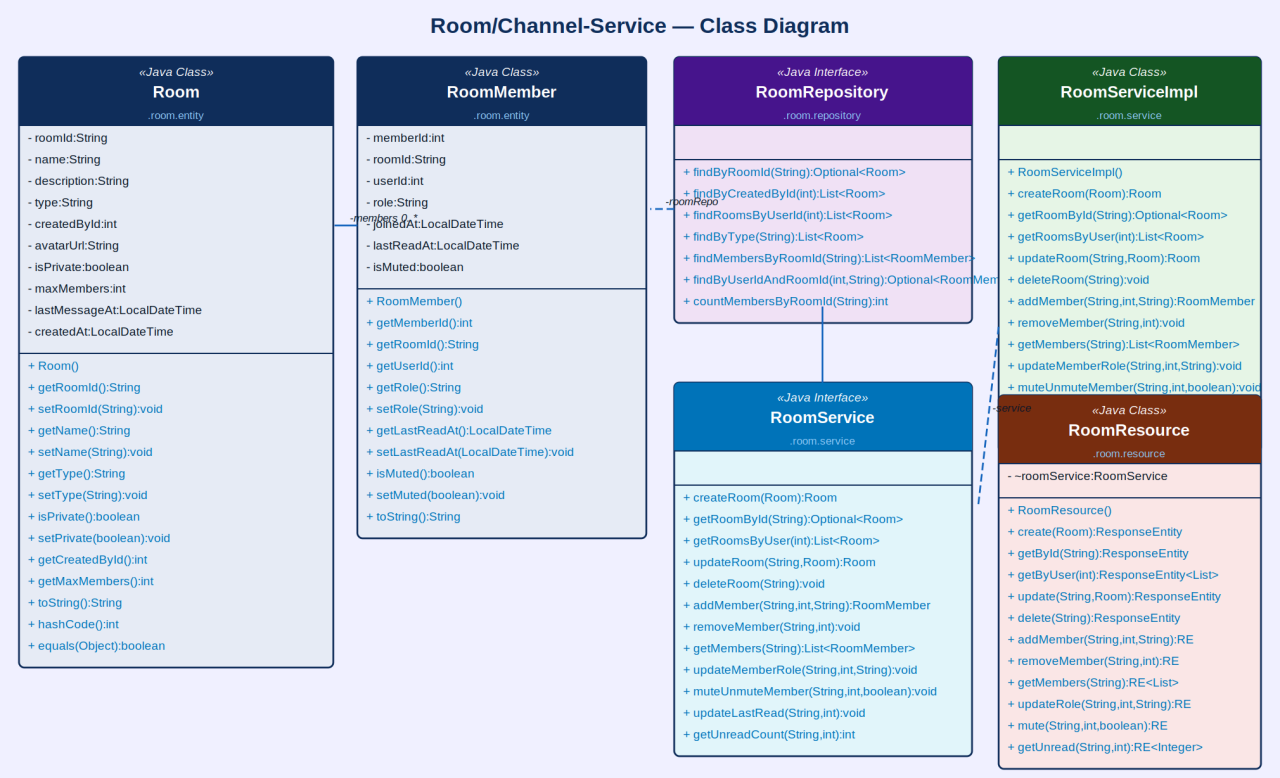

Figure 3: Room/Channel-Service — Class Diagram

Component	Type	Key Attributes / Methods
Room	Java Class (Entity)	roomId (UUID), name, description, type (GROUP/DM), createdById, avatarUrl, isPrivate, maxMembers, lastMessageAt, createdAt
RoomMember	Java Class (Entity)	memberId, roomId, userId, role (ADMIN/MEMBER), joinedAt, lastReadAt (for unread tracking), isMuted — linked to Room (0..*)
RoomRepository	Java Interface	findByRoomId(), findByCreatedById(), findRoomsByUserId(), findByType(), findMembersByRoomId(), findByUserIdAndRoomId(), countMembersByRoomId()

Component	Type	Key Attributes / Methods
RoomServiceImpl	Java Class	createRoom(), getRoomById(), getRoomsByUser(), updateRoom(), deleteRoom(), addMember(), removeMember(), getMembers(), updateMemberRole(), muteUnmuteMember(), updateLastRead(), getUnreadCount()
RoomService	Java Interface	Declares all room CRUD, member management, role/mute control, last-read tracking, and unread count operations
RoomResource	Java Class (REST)	Exposes /rooms endpoints: POST (create), GET (by id/user), PUT (update/mute/role), DELETE, POST/DELETE (members), GET (unread count)

4.3 Message-Service

The Message-Service provides persistence and retrieval for all chat messages. Messages are persisted when the ChatWebSocketHandler receives a CHAT_MESSAGE STOMP frame — the handler calls sendMessage() and then broadcasts the saved Message object (with its generated messageId) back to /topic/room/{roomId}. Pagination is implemented using Spring Data's Pageable to support infinite scroll. The deliveryStatus field transitions SENT → DELIVERED → READ driven by STOMP events.

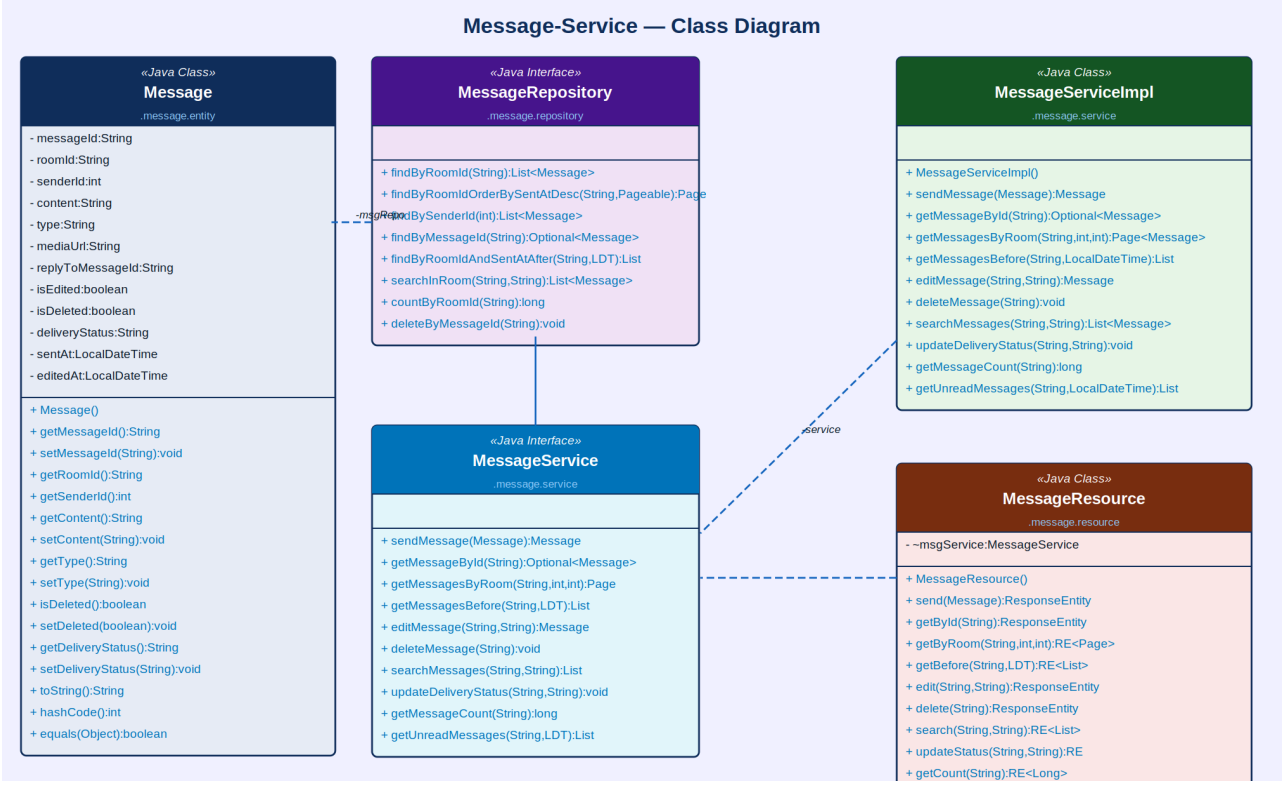

Figure 4: Message-Service — Class Diagram

Component	Type	Key Attributes / Methods
Message	Java Class (Entity)	messageId (UUID), roomId, senderId, content, type (TEXT/IMAGE/FILE/REACTION/SYSTEM), mediaUrl, replyToMessageId, isEdited, isDeleted, deliveryStatus (SENT/DELIVERED/READ), sentAt, editedAt
MessageRepository	Java Interface	findByRoomId(), findByRoomIdOrderBySentAtDesc(), findBySenderId(), findByMessageId(), findByRoomIdAndSentAtAfter(), searchInRoom(), countByRoomId(), deleteByMessageId()

Component	Type	Key Attributes / Methods
MessageServiceImpl	Java Class	sendMessage(), getMessageById(), getMessagesByRoom(), getMessagesBefore(), editMessage(), deleteMessage(), searchMessages(), updateDeliveryStatus(), getMessageCount(), getUnreadMessages()
MessageService	Java Interface	Declares all message persistence, pagination, editing, soft-deletion, search, delivery status update, and unread retrieval operations
MessageResource	Java Class (REST)	Exposes /messages endpoints: POST (send), GET (by id/room/before), PUT (edit), DELETE, GET (search/count/unread), PUT (status)

4.4 Media / File-Service

The Media/File-Service handles all file uploads before they are shared in chat. The upload flow is: client makes a multipart REST POST to /media/upload, the file is stored on AWS S3, a thumbnail is generated server-side for images using an image processing library (Thumbnailator), and the returned MediaFile entity (containing url and thumbnailUrl) is embedded in the subsequent STOMP CHAT_MESSAGE frame. Pre-signed S3 URLs with 24-hour expiry are used for downloads.

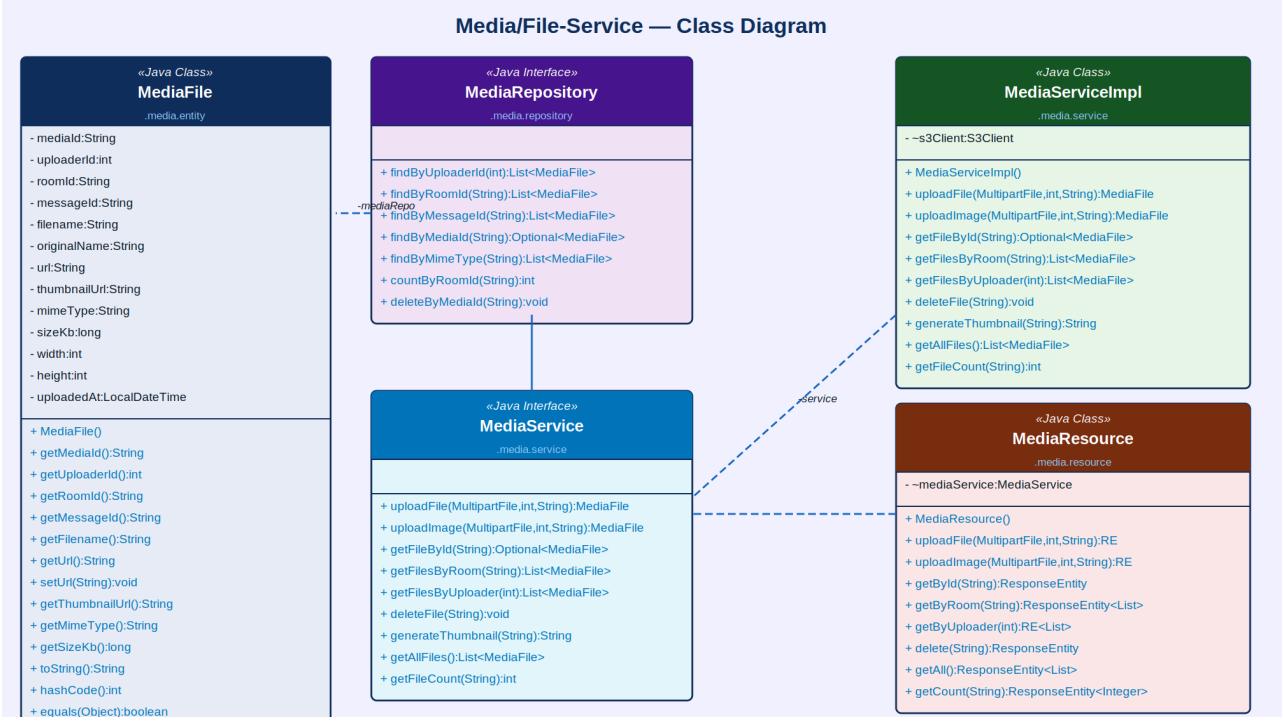

Figure 5: Media/File-Service — Class Diagram

Component	Type	Key Attributes / Methods
MediaFile	Java Class (Entity)	mediaId (UUID), uploaderId, roomId, messageId, filename, originalName, url (S3), thumbnailUrl, mimeType, sizeKb, width, height, uploadedAt
MediaRepository	Java Interface	findByUploaderId(), findByRoomId(), findByMessageId(), findByMediaId(), findByMimeType(), countByRoomId(), deleteByMediaId()
MediaServiceImpl	Java Class	uploadFile(), uploadImage(), getFileById(), getFilesByRoom(), getFilesByUploader(), deleteFile(), generateThumbnail(), getAllFiles(), getFileCount()
MediaService	Java Interface	Declares multipart upload (file/image), S3 storage, thumbnail generation, retrieval by room/uploader, deletion, and count operations

Component	Type	Key Attributes / Methods
MediaResource	Java Class (REST)	Exposes /media endpoints: POST (uploadFile/uploadImage), GET (by id/room/uploader/all), DELETE, GET count

4.5 Presence / Status-Service

The Presence/Status-Service maintains the live online status of all users using a Redis-backed store for low-latency reads. When the ChatWebSocketHandler detects a new WebSocket connection (afterConnectionEstablished), it calls setOnline() with the userId, device type, and session ID. On disconnect (afterConnectionClosed), it calls setOffline(). A session ping mechanism updates lastPingAt every 30 seconds; a scheduled job runs every 60 seconds to clean sessions that have not pinged (stale session cleanup). Bulk presence reads are used to populate the member list UI.

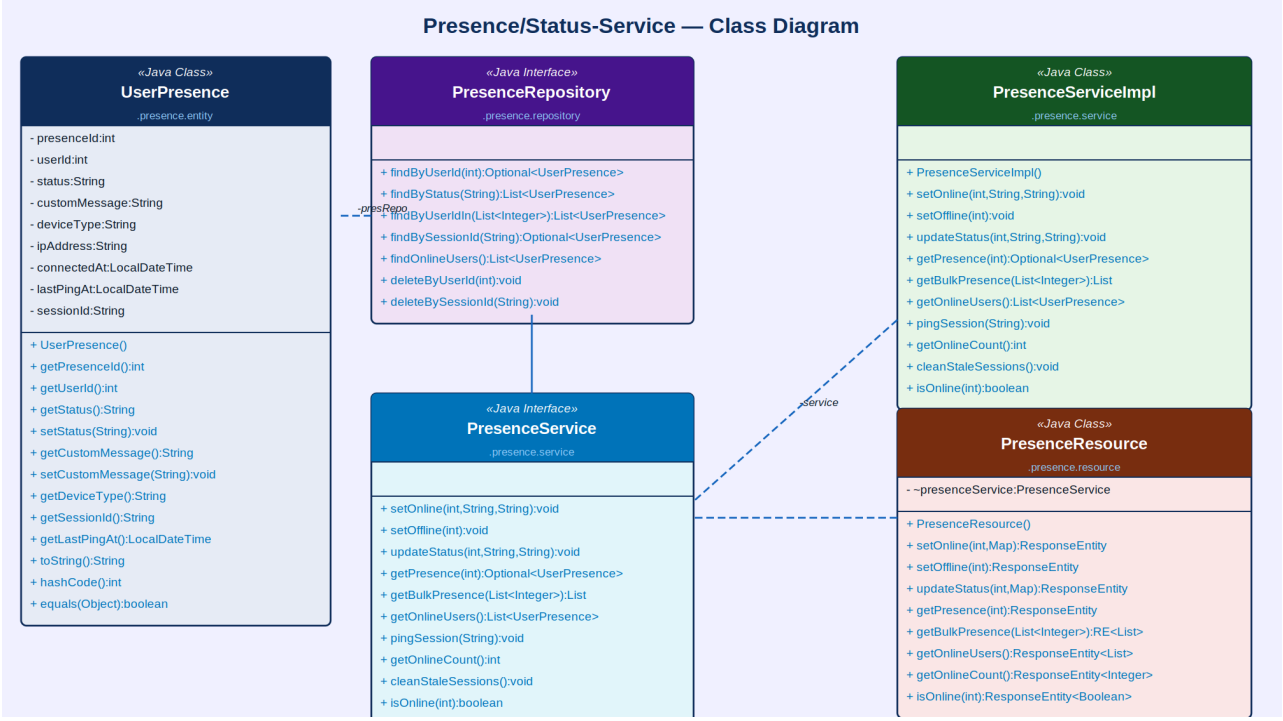

Figure 6: Presence/Status-Service — Class Diagram

Component	Type	Key Attributes / Methods
UserPresence	Java Class (Entity)	presenceId, userId, status (ONLINE/AWAY/DND/INVISIBLE), customMessage, deviceType, ipAddress, connectedAt, lastPingAt, sessionId
PresenceRepository	Java Interface	findByUserId(), findByStatus(), findByUserIdIn(), findBySessionId(), findOnlineUsers(), deleteByUserId(), deleteBySessionId()
PresenceServiceImpl	Java Class	setOnline(), setOffline(), updateStatus(), getPresence(), getBulkPresence(), getOnlineUsers(), pingSession(), getOnlineCount(), cleanStaleSessions(), isOnline()

Component	Type	Key Attributes / Methods
PresenceService	Java Interface	Declares online/offline lifecycle, status updates, bulk presence reads, session ping, stale cleanup, and online count operations
PresenceResource	Java Class (REST)	Exposes /presence endpoints: POST (setOnline/setOffline), PUT (status), GET (single/bulk/all/count), GET isOnline

4.6 Notification-Service

The Notification-Service dispatches in-app, push (Firebase FCM), and email notifications for offline users. In-app notifications are delivered immediately via the personal WebSocket queue (/topic/user/{userId}) when the recipient is connected. For offline recipients, a Firebase Cloud Messaging push notification is dispatched. Email notification is sent for missed DMs when the recipient has been offline for more than 30 minutes.

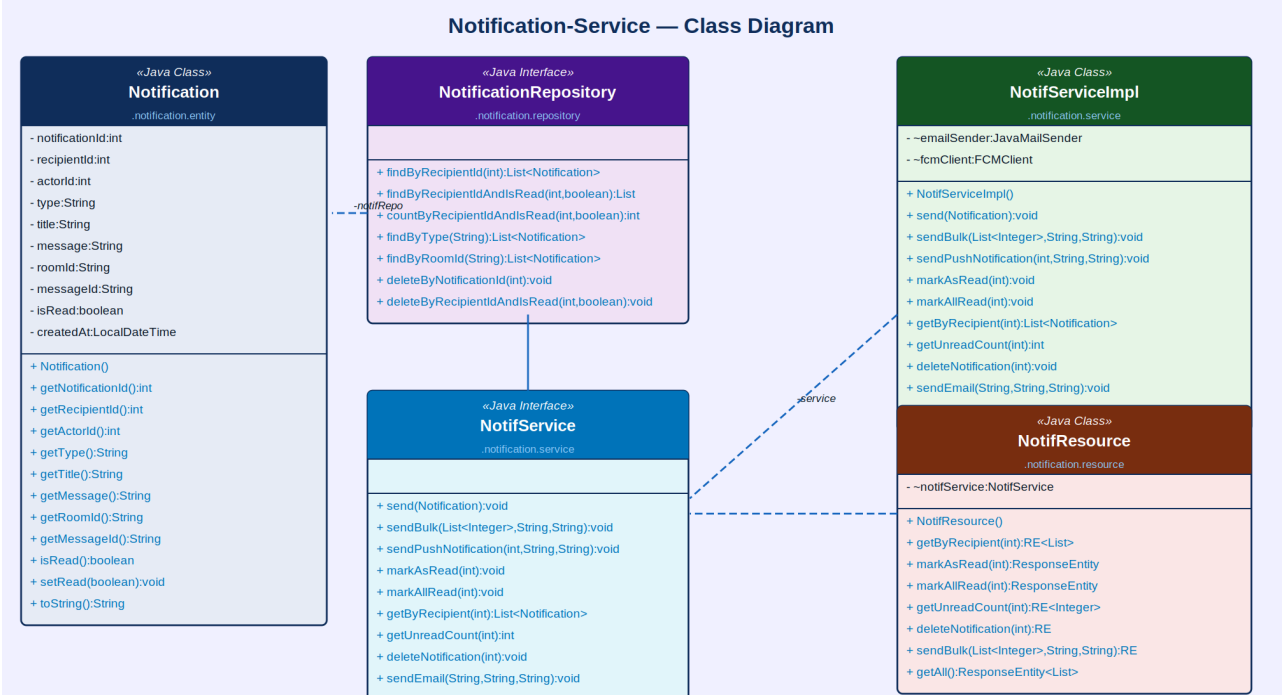

Figure 7: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, actorId, type (NEW_MESSAGE/MENTION/ROOM_INVITE/SYSTEM), title, message, roomId, messageId, isRead, createdAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRoomId(), deleteByNotificationId(), deleteByRecipientIdAndIsRead()
NotifServiceImpl	Java Class	send(), sendBulk(), sendPushNotification(), markAsRead(), markAllRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), getAll()
NotifService	Java Interface	Declares in-app alert dispatch, FCM push notification, bulk send, read-state management, and email fallback operations

Component	Type	Key Attributes / Methods
NotifResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, getUnreadCount, delete, sendBulk, getAll

4.7 WebSocket Handler — ChatWebSocketHandler & WebSocketConfig

The WebSocket Handler layer is the real-time messaging core of ConnectHub — the Java equivalent of Socket.io's event emitter. ChatWebSocketHandler implements Spring's WebSocketHandler interface and manages a ConcurrentHashMap of active WebSocket sessions keyed by sessionId. WebSocketConfig implements WebSocketMessageBrokerConfigurer to configure the STOMP message broker with /app as the application prefix and /topic as the broker prefix, enabling Spring to route STOMP frames directly to subscribed clients without custom relay logic.

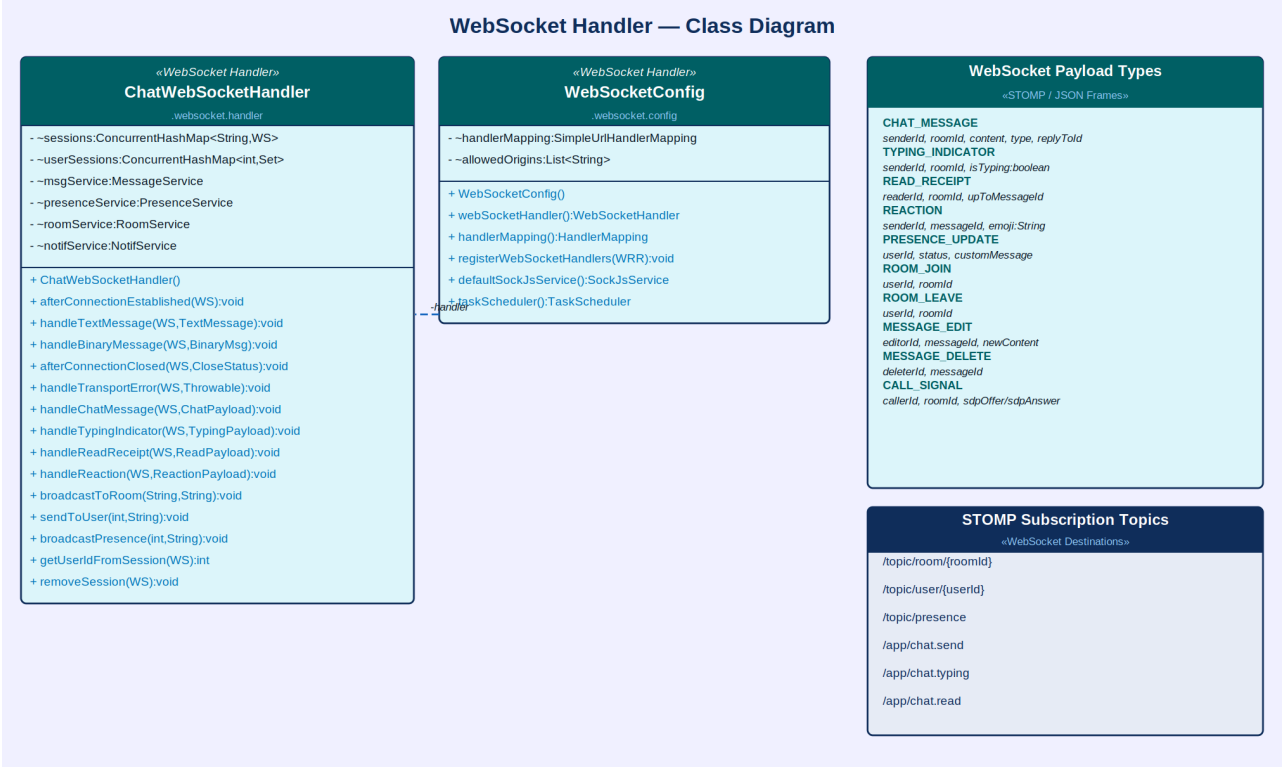

Figure 8: WebSocket Handler — Class Diagram with STOMP Payload Types and Subscription Topics

Component	Type	Key Attributes / Methods
ChatWebSocketHandler	WebSocket Handler (Spring)	Manages ConcurrentHashMap of sessions; handles afterConnectionEstablished (sets presence online), handleTextMessage (routes by type: CHAT/TYPING/READ/REACTION), handleTransportError, afterConnectionClosed (sets presence offline). Implements broadcastToRoom(), sendToUser(), broadcastPresence()
WebSocketConfig	Spring Configuration	Implements WebSocketMessageBrokerConfigurer; configures STOMP broker on /topic and /queue prefixes; registers /app as message prefix; enables SockJS fallback at /ws endpoint; configures TaskScheduler for heartbeat

Component	Type	Key Attributes / Methods
STOMP Payload Types	JSON Frame Schemas	CHAT_MESSAGE (senderId, roomId, content, type, replyToId), TYPING_INDICATOR (senderId, roomId, isTyping), READ_RECEIPT (readerId, roomId, upToMessageId), REACTION (senderId, messageId, emoji), PRESENCE_UPDATE (userId, status), MESSAGE_EDIT (editorId, messageId, newContent), MESSAGE_DELETE (deleterId, messageId)
STOMP Subscription Topics	WebSocket Destinations	/topic/room/{roomId} — all room messages and events; /topic/user/{userId} — personal alerts and DM notifications; /topic/presence — online/offline presence broadcasts; /app/chat.send, /app/chat.typing, /app/chat.read — inbound message endpoints

4.8 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC REST and view layer that handles HTTP requests (authentication, room creation, message history, media upload) while the ChatWebSocketHandler handles all real-time STOMP frames. Three controllers are defined: ChatController (user-facing views and REST endpoints), RoomManagerController (room settings and moderation), and AdminController (admin panel). REST and WebSocket endpoints co-exist on the same server — HTTP for persistent data operations, WebSocket for real-time event delivery.

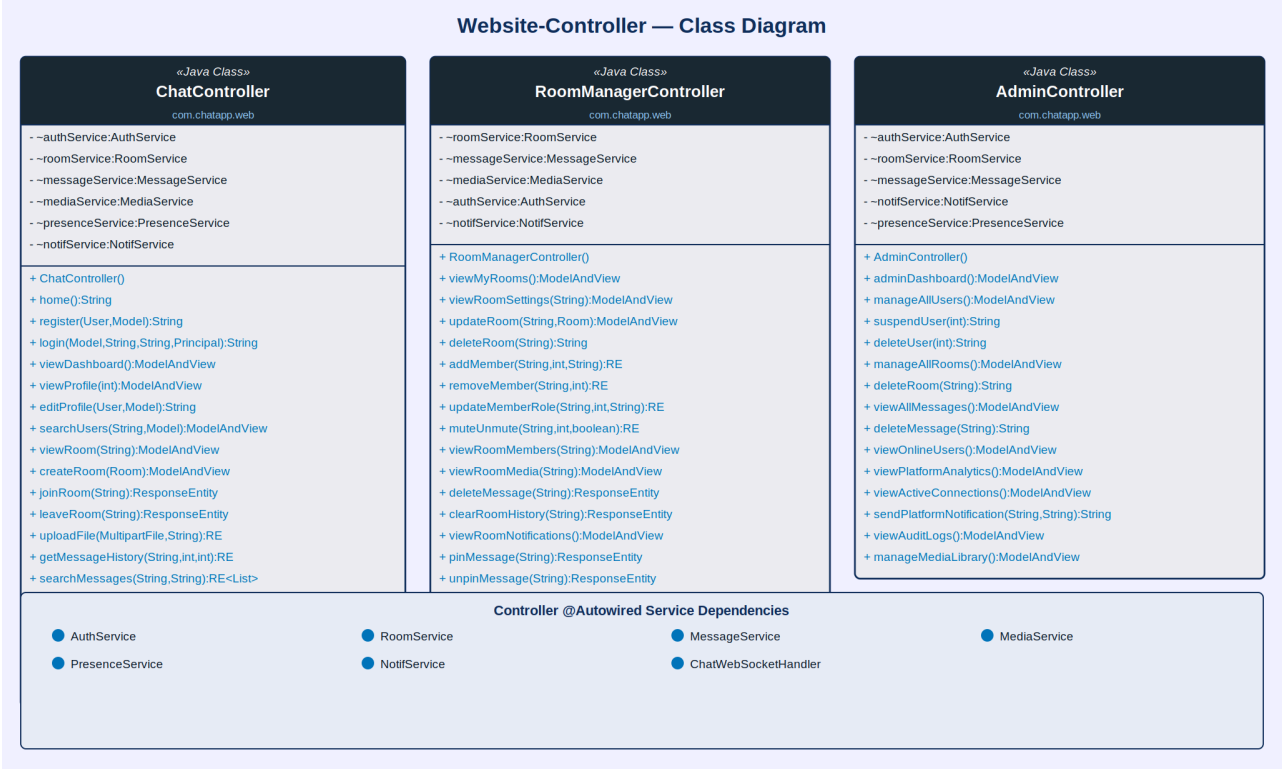

Figure 9: Website-Controller — Class Diagram

Controller	Type	Key Methods
ChatController	Java Class (MVC)	home(), register(), login(), viewDashboard(), viewProfile(), editProfile(), searchUsers(), viewRoom(), createRoom(), joinRoom(), leaveRoom(), uploadFile(), getMessageHistory(), searchMessages(), viewNotifications(), markNotifRead(), getPresence(), updateStatus(), logout()
RoomManagerController	Java Class (MVC)	viewMyRooms(), viewRoomSettings(), updateRoom(), deleteRoom(), addMember(), removeMember(), updateMemberRole(), muteUnmute(), viewRoomMembers(), viewRoomMedia(), deleteMessage(),

Controller	Type	Key Methods
		clearRoomHistory(), viewRoomNotifications(), pinMessage(), unpinMessage()
AdminController	Java Class (MVC)	adminDashboard(), manageAllUsers(), suspendUser(), deleteUser(), manageAllRooms(), deleteRoom(), viewAllMessages(), deleteMessage(), viewOnlineUsers(), viewPlatformAnalytics(), viewActiveConnections(), sendPlatformNotification(), viewAuditLogs(), manageMediaLibrary()

5. Microservices Architecture Overview

ConnectHub is structured as a set of independently deployable microservices following the EShoppingZone reference pattern. Each service owns its own database schema, REST API contract, service interface, and business logic. The WebSocket Handler layer runs within the same Spring Boot application as the REST controllers but operates on a separate thread pool managed by the TaskScheduler. Redis is used for Presence-Service low-latency reads and the STOMP session registry.

Microservice / Layer	Base Package	Primary Domain
auth-service	com.connecthub.auth	User accounts, JWT, OAuth2 (Google/GitHub), status, last-seen
room-service	com.connecthub.room	Room CRUD, membership, roles, mute, last-read, unread count
message-service	com.connecthub.message	Message persistence, pagination, edit/delete, delivery status, search
media-service	com.connecthub.media	File/image upload to S3, thumbnail generation, room media gallery
presence-service	com.connecthub.presence	Live online status, session tracking, bulk presence reads, stale cleanup
notification-service	com.connecthub.notification	In-app alerts, FCM push, email fallback, mention notifications
websocket-handler	com.connecthub.websocket	STOMP broker config, WebSocket session management, real-time routing
connecthub-web	com.connecthub.web	Spring MVC controllers, Thymeleaf / React SPA view rendering

6. Non-Functional Requirements

Category	Requirement
Real-time Latency	Message delivery latency under 100ms for users on the same server instance; Redis Pub/Sub distributes STOMP broadcasts across multiple server instances for horizontal scale
Scalability	WebSocket connections are sticky-session load balanced; Redis Pub/Sub ensures STOMP frames are broadcast across all server instances in the cluster
Security	JWT token required in STOMP CONNECT Authorization header; tokens validated on every WebSocket upgrade; all messages are TLS-encrypted; message content sanitised to prevent XSS via STOMP payloads
Concurrency	ConcurrentHashMap manages WebSocket sessions thread-safely; message broadcast uses CompletableFuture for non-blocking delivery to multiple sessions
Availability	99.9% uptime SLA; SockJS fallback (long-polling) activates automatically when WebSocket is blocked by proxies; health-check endpoints per service
Session Management	Stale WebSocket sessions (no ping in 60 seconds) are cleaned by a Spring @Scheduled job; presence is set to offline on cleanup
Typing Indicator	Typing events are not persisted; they are broadcast only to currently connected room subscribers with a 3-second client-side debounce to prevent event flooding
Message Ordering	Messages are stored with sentAt (server-side timestamp) ensuring consistent ordering across all clients regardless of client clock differences
Media Limits	Max file size 25MB; image types: JPEG, PNG, GIF, WebP; document types: PDF, DOCX, XLSX, ZIP; enforced at API Gateway before S3 upload
Audit Trail	All admin actions and room moderation events (delete message, remove member, clear history) logged with actor, timestamp, and entity reference

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA, Spring WebSocket)
Real-time Protocol	STOMP over WebSocket via Spring's WebSocketMessageBrokerConfigurer; SockJS fallback for browsers behind proxies
WebSocket Alternative	Spring WebSocket + STOMP replaces Socket.io: @MessageMapping annotations route inbound frames; SimpMessagingTemplate broadcasts outbound frames
Authentication	JWT (JSON Web Tokens) passed as STOMP CONNECT header; Spring Security WebSocket interceptor validates JWT on handshake
Database	MySQL (message and room persistence); Redis (presence store, STOMP session registry, Pub/Sub for multi-instance broadcast)
Redis Pub/Sub	Redis Pub/Sub channel per room distributes STOMP broadcast messages across all horizontally-scaled server instances
File Storage	AWS S3 for media uploads; pre-signed URLs for download; CloudFront CDN for thumbnail delivery
Image Processing	Thumbnailator (Java) for server-side image thumbnail generation on upload
Push Notifications	Firebase Cloud Messaging (FCM) Java SDK for mobile push to offline users
Email	Spring JavaMailSender via AWS SES for missed DM email notifications
Frontend	React.js SPA with SockJS + @stomp/stompjs client library for WebSocket connection; or Thymeleaf with native WebSocket API
Containerisation	Docker with Docker Compose; Kubernetes with sticky-session Ingress for WebSocket load balancing
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints; WebSocket frame schemas documented in AsyncAPI 2.6
CI/CD	GitHub Actions pipeline: lint → test → Docker build → ECR push → Kubernetes rolling deploy with WebSocket drain

8. Glossary

Term	Definition
WebSocket	A full-duplex TCP-based communication protocol that maintains a persistent connection between client and server, enabling real-time bidirectional data exchange
STOMP	Simple Text Oriented Messaging Protocol — a frame-based messaging protocol layered over WebSocket that defines message types: CONNECT, SUBSCRIBE, SEND, MESSAGE, and DISCONNECT
SockJS	A JavaScript library providing a WebSocket-like API with automatic fallback to HTTP long-polling when WebSocket is unavailable or blocked by a proxy
STOMP Broker	Spring's built-in in-memory message broker (or RabbitMQ/ActiveMQ as external broker) that routes STOMP frames from /app destinations to /topic subscribers
Session	A unique identifier for a connected WebSocket client stored in the server's ConcurrentHashMap and used for targeted message delivery
Topic	A STOMP subscription destination (e.g., /topic/room/{roomId}) to which multiple clients subscribe; any message published to the topic is broadcast to all subscribers
Typing Indicator	A transient STOMP event broadcast when a user is actively typing; not persisted to the database; cleared automatically after 3 seconds of inactivity
Read Receipt	A STOMP event broadcast when a user reads messages up to a specific messageId, enabling per-user READ delivery status on messages
Delivery Status	A message field transitioning through SENT (persisted) → DELIVERED (recipient session active) → READ (read receipt received)
Presence	A real-time record of a user's connection state (ONLINE/AWAY/DND/INVISIBLE) and last-seen timestamp stored in Redis for fast lookup
DM	Direct Message — a private 1:1 room automatically created between exactly two users with type=DM and isPrivate=true
Stale Session	A WebSocket session that has not sent a ping in over 60 seconds; cleaned by the scheduled Presence-Service job, setting the user to offline
SimpMessagingTemplate	Spring's high-level STOMP messaging abstraction that simplifies broadcasting to topics and user-specific queues from any Spring component
Pre-signed URL	A time-limited AWS S3 URL that grants temporary read access to a private file without requiring AWS credentials
Heartbeat	A periodic STOMP PING/PONG exchange configured in WebSocketConfig to detect and close dead connections

Term	Definition
Socket.io Alternative	This platform uses Spring WebSocket + STOMP + SockJS as the Java server-side equivalent of Node.js Socket.io's real-time event emitter architecture